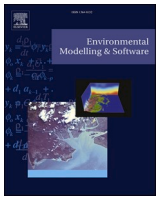
Environmental Modelling and Software 201 (2026) 106958



Contents lists available at ScienceDirect

Environmental Modelling and Software

journal homepage: www.elsevier.com/locate/envsoft





Converging human intelligence with AI systems to advance flood evacuation decision

Rishav Karanjit [a], Vidya Samadi [b,c,*], Amanda L. Hughes [d], Pamela Murray-Tuite [e], Keri K. Stephens [f]

[a] School of Computing, Clemson University, SC, USA
[b] Department of Agricultural Sciences, Clemson University, SC, USA
[c] Artificial Intelligence Research Institute for Science and Engineering (AIRISE), Clemson University, SC, USA
[d] Computer Science Department, Brigham Young University, UT, USA
[e] The Glenn Department of Civil Engineering, Clemson University, SC, USA
[f] Moody College of Communication, The University of Texas at Austin, TX, USA

ARTICLE INFO

Keywords:
Artificial intelligence
Human-AI convergence
Flood emergency management
Evacuation decision making

ABSTRACT

The powers that artificial intelligence (AI) have developed are impressive, with recent success in leveraging human expertise at various stages of model development. However, AI can only attain its full potential when it actively teams with humans to co-create solutions. This study introduces a Human-AI Convergence (HAC) system for flood evacuation decision-making. The goal is to develop a unique, computationally effective HAC system for flood evacuation decision-making that integrates AI with transportation geospatial data, a river hydraulic model, and human data from X. The HAC system is smartly designed to forecast flood stage levels using AI across the US Geological Survey gauging stations, visualized in a web-based Google Earth architecture. HAC has been tested in the Lowcountry of South Carolina, where previous floods caused considerable damage to the transportation networks and increased traffic on evacuation routes. The system stands to advance the frontier of human-AI collaboration for real-time flood management.

1. Introduction

Evacuation is crucial for minimizing the risk of injury or loss of life during flooding events. However, the decision to evacuate can be complex, involving multiple factors including social considerations, resource availability, isolation of location, and capacity of the infrastructure (Kolen et al., 2013). The costs of an evacuation in the case of hurricanes in the United States can exceed 1 million dollars per mile due to losses in commerce, productivity, and direct losses to goods (Wolshon et al., 2005). To reduce this cost, deterministic models such as a heuristically driven flood evacuation planning model (Bennett et al., 2017) and stochastic models such as a Fuzzy logic-based decision support system (Jia et al., 2016) have been developed to aid decision-makers in planning and preparing for flood evacuation processes. However, when a flood disaster occurs, analyzing complex information and data to make quick evacuation decisions is a challenging task for decision-makers and authorities. Therefore, providing more rapid and accurate hazard modeling is essential for a safe and smart emergency response and decision-making.

Machine learning approaches are increasingly becoming a viable solution for flood evacuation decisions. Scholars have recently developed machine learning models to forecast flooding and determine safe evacuation routes during emergencies (Sreejith et al., 2022; Wang et al., 2023). The results of these studies are important for rapid evacuation decision-making, however, a mechanism to incorporate decision-makers' knowledge and data into machine-learning approaches is lacking. The approach of uniting data from humans and machine learning leverages the strengths of humans and machine learning systems, resulting in more efficient and effective flood evacuation decisions. Indeed, combining machine learning simulation with human understanding and strategic abilities through data convergence may optimize the flood evacuation process and provide a capacity akin to "teaming intelligence". In this human-artificial intelligence convergence (HAC) system, humans can perform tasks such as search and rescue, communication, and flood damage validation, which require human knowledge and social skills while machine learning can perform flood

* Corresponding author. Department of Agricultural Sciences, Clemson University, SC, USA.
E-mail address: samadi@clemson.edu (V. Samadi).

https://doi.org/10.1016/j.envsoft.2026.106958
Received 24 June 2025; Received in revised form 27 January 2026; Accepted 11 March 2026
Available online 24 March 2026
1364-8152/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

forecasting and analyze massive real-time data and information. This cooperation is driven by a shared objective, which necessitates exchanging crucial information through diverse forms of communication, prediction, and the achievement of high-level coordination tasks (McNeese et al., 2018).

Previous studies related to concepts similar to HAC have explored a range of applications, including the interaction and effectiveness of human teams with AI in robotic swarms (Seeber et al., 2020), perimeter security for landed aircraft (Madni and Madni, 2018), collaborative gaming environments (Ong et al., 2012), and the identification of risky human behavior (Stephens et al., 2023). Other research has examined how factors such as a person's understanding of the limitations or potential errors of a machine learning system can influence team performance within HAC frameworks (Bansal et al., 2019; Liang et al., 2019). Samadi et al. (2024) provided a comprehensive review of human-AI teaming strategies designed to enable effective collaboration during flood emergencies, highlighting approaches for seamless integration into existing workflows and supporting more informed, data-driven decision-making. Collectively, these studies have fueled growth in HAC literature, emphasizing scenarios where humans and AI converge to perform complex tasks as integrated teams. However, despite these advancements, HAC has not yet been applied to flood response and evacuation challenges, a gap that this study aims to address.

The goal of this study is to address this knowledge gap by examining and synthesizing HAC competencies in flood evacuation decision-making, leveraging the potential of machine learning as an active partner in real-time decisions. In this context, “real-time” refers to making decisions with minimal delay, typically based on National Weather Service (NWS) forecasts available 3–5 days in advance. This research examines the step-by-step structure of employing a HAC system for flood evacuation planning in South Carolina, USA. The intention is not to include all possible algorithms, applications, and techniques, but rather to provide case study applications where a HAC system has been successfully implemented. As part of this study, a HAC system was developed for flood evacuation decision-making. This system serves as a proof of concept to illustrate how diverse data streams can be unified into a single workflow and how HAC can enrich situational awareness. Rather than presenting a fully operational system, this work provides a general structure for researchers to apply HAC concepts, while also evaluating the state-of-the-art in this area and outlining a roadmap for future studies aimed at improving performance, addressing uncertainties, and advancing practical adoption. Our developed HAC system combines machine learning models with human data to predict flood depth and inundation areas and then use these forecasts to determine flood evacuation rerouting decisions. The system includes machine learning approaches for forecasting floods across the US Geological Survey (USGS) gauging stations and incorporates the results into a Height Above Nearest Drainage (HAND) model to calibrate inundation areas in real-time. In addition, we leveraged human data into the HAC system by integrating X (previously Twitter) data into the system. In this integrated HAC framework, the fusion of flood level predictions, a river hydraulic model, transportation data, and real-time X observations heralds an innovative paradigm in flood evacuation prediction and response strategies.

The uniqueness of the proposed Human–AI Convergence (HAC) system lies in how human and artificial intelligence (AI) complement and refine each other's roles during flood evacuation decision-making, rather than merely fusing independent data streams. In the HAC framework, AI modules (e.g., LSTM flood forecasts, HAND inundation mapping, and rerouting algorithms) generate predictive and spatially explicit information, while human-generated inputs from X provide real-time, ground-level validation of these outputs. This validation layer allows decision-makers to reconcile AI-predicted flood zones with on-the-ground conditions and adjust evacuation routes accordingly. Thus, convergence occurs at the decision interface—AI enhances human situational awareness through predictive analytics, and humans enhance AI-driven outputs by confirming, rejecting, or prioritizing them based on lived context. Conceptually, the HAC system functions through a bidirectional feedback loop, where model outputs inform human actions, and human observations continuously enrich the situational dataset used for subsequent analyses. This cooperative cycle of prediction to validation to adjustment distinguishes HAC from simple data fusion, positioning it as a genuinely convergent human–AI decision ecosystem.

The remainder of this paper is structured as follows: Section 2 provides an introduction to the machine learning models, human data, river hydraulic model, rerouting approach, HAC workflow, and performance metrics. Section 3 presents the results of the HAC applications. Finally, Section 4 presents the discussion, conclusions, and paths for future research.

2. Methodology

2.1. Study area and data

Our HAC system was developed and tested for the Lowcountry in South Carolina (SC), USA where frequent flooding caused significant damage to critical infrastructure, properties, and people's lives. The Lowcountry is characterized by a low elevation, flat terrain area prone to inundation conditions and storm surge. Following recent major flooding such as the SC Flood of 2015 and Hurricane Matthew in 2016, many local roads in the Lowcountry were under water, which limited mobility, hampered evacuation response, and, in some places, isolated communities. Roads in this region were not built high enough to accommodate water flowing around and under them (Phillips, 2020). Consequently, managing flood damage and facilitating evacuations pose major challenges in this region.

We tested the HAC system for multiple USGS gauging stations in the Lowcountry, as case studies (see Fig. 1). Rainfall and river data were collected from the USGS and NWS. We trained the machine learning models for three USGS gauging stations located in the Lowcountry, including Turkey Creek (USGS02172035), South Fork Edisto River (USGS02173000), and North Fork Edisto River (USGS02173500), shown in Fig. 1.

Historical time series data of precipitation and gauge height obtained from the USGS were used to train machine learning algorithms. During non-flooding events, the gauge height of the river was slow-changing. Conversely, gauge height values changed significantly during flooding events over short intervals of 15 min. Since the flood prediction task was defined on an hourly basis, we used the pandas (McKinney, 2010) library to calculate the cumulative daily data. We used machine learning algorithms, NWS rainfall forecast data, and a Rational method along with a rating curve conversion tool (explained in the next sections) to predict flood level (or gauge height of the river) in ungauged/poorly gauged watersheds in the Lowcountry, SC.

2.2. Machine learning algorithms

We trained two types of recurrent neural network (RNN) models, i.e., Long Short-Term Memory (LSTM; Hochreiter and Schmidhuber, 1997) and Gated Recurrent Unit (GRU; Cho, 2014) using three USGS gauging stations. For each station, we used Optuna (Akiba et al., 2019) to optimize the hyperparameters. We trained 30 models for each station (overall 180 models [2 machine learning algorithms, 3 stations and 30 models each with different parameters]) and used the best model for real-time flood forecasting. During training, we used the pruning technique in Optuna as an early stopping technique. Pruning within the Optuna hyperparameter optimization library stops RNN training early if it is deemed unlikely to produce a better result than the previous best-known model configuration without sacrificing the quality of results (Akiba et al., 2019). From the 180 models produced during training for all three stations, the six best models were selected from the Optuna

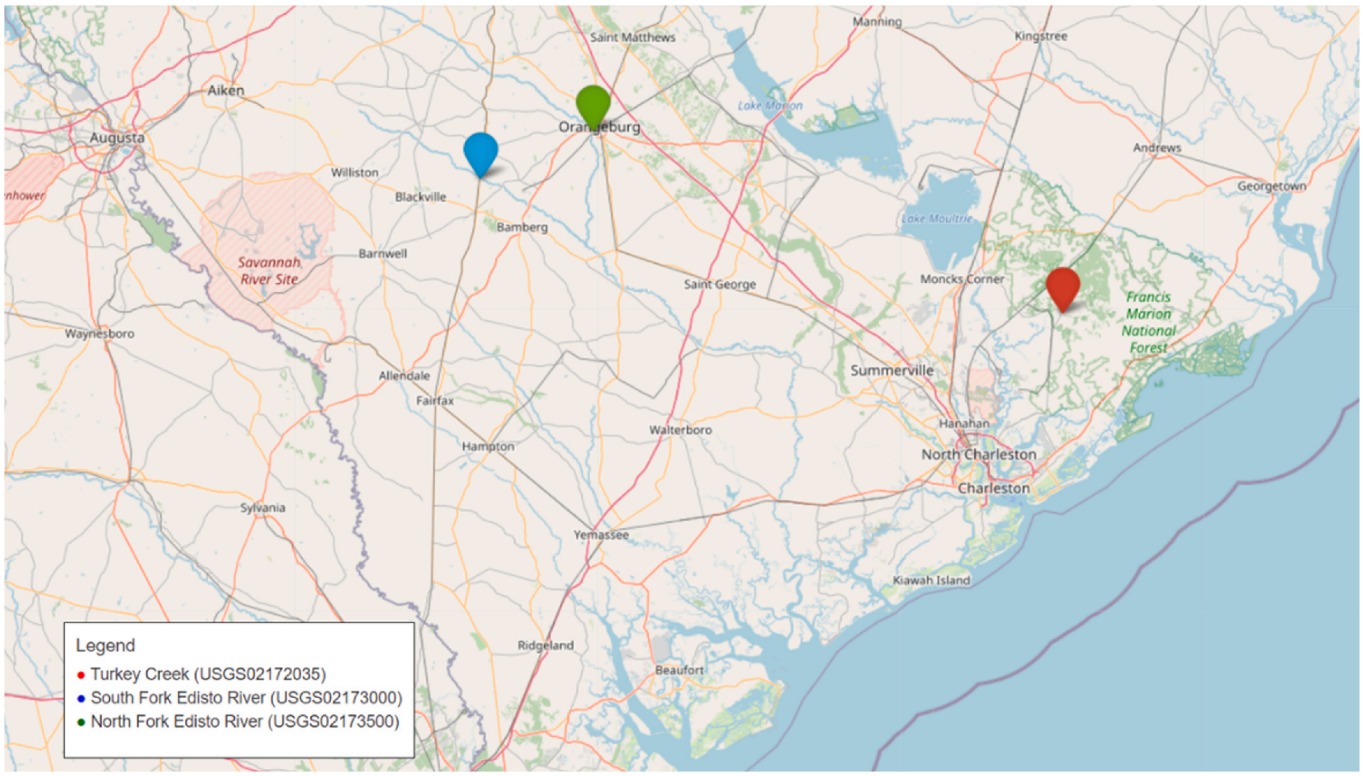


Fig. 1. The USGS gauging stations in the Lowcountry, SC used in this research.

library (for three gauging stations and two models), and then the three best models were chosen manually - one for each station based on performance metrics.

The architecture of LSTM and GRU variants were specifically modified to address the issue of vanishing gradients that are commonly encountered in conventional recurrent networks. LSTM is equipped with a specialized memory cell capable of retaining information for extended periods. Additionally, this network features three distinct types of gates - namely, the input gate, forget gate, and output gate - which regulate the inflow and outflow of information to and from the memory cell. The gates incorporated in the network facilitate the selective retention or omission of information, rendering it highly appropriate for applications that entail the manipulation and retention of sequential data, such as Natural Language Processing (NLP), speech recognition, and time series prediction. The LSTM network receives input data as a sequential vector set, with each individual LSTM unit processing a single vector at each time step. The output of each LSTM unit is a hidden state vector that is subsequently utilized as input for the following time step. The LSTM model can effectively model intricate sequential data by using gates to regulate the flow of information within the network. This enables the model to retain information from past inputs and leverage it to make informed predictions about future inputs.

Our study employed an LSTM consisting of six layers, a dropout value, and a dense layer. The first three hidden layers were followed by a dropout layer, which was then followed by the remaining LSTM layers. This was succeeded by a flattened layer and a dense layer containing five neurons. The spatial dimensions of the input are reduced to the size of the channel by a flattened layer. The LSTM layer is designed to predict the subsequent 5 data points (5 h in advance) by utilizing the preceding 48 data points (48 h) as “look back” time. The 48-h look-back window captures two full diurnal rainfall-runoff cycles, ensuring that delayed hydrological responses and cumulative precipitation effects are represented. Shorter windows (e.g., 12–24 h) underperformed due to the absence of lagged flood peaks. The 5-h forecast horizon reflects the average decision window used by state emergency management (based on NWS alerts), balancing predictive accuracy and operational usability. Initially, LSTM showed overfitting in terms of low gauge height value prediction. This network, with its ability to capture long-term dependencies, is prone to overfitting, especially when the data values are small. To mitigate this issue, we implemented Optuna as an early stopping technique to monitor the model performance during the validation period and stop the training process when the performance began to degrade.

While Optuna allowed us to implement state-of-the-art optimization algorithms to speed up the hyperparameter tuning process, these advanced algorithms are built to efficiently search for the best objective when the cost to iterate the model training process is too expensive. If we train our models with a small amount of data, it is possible that Optuna uses Random Search and Grid Search for hyperparameter tuning, which can either spend too much time or cannot locate the minimum. On the other hand, if the data volume increases and models get more complex, the cost of using Random Search and Grid Search to train a set of hyperparameters increases significantly. The dropout rate, number of units for each layer and epoch number were decided after training 180 models using Optuna. This is because the quantity of water flowing into and out of a river system affects the height of a gauge. The river system receives runoff from precipitation, whereas the runoff output from the system is represented by discharge or gauge height. Briefly, the gauge height of the river was predicted using LSTM and GRU models, the best model was then selected, which was the LSTM, for forecasting gauge height in real-time using NWS rainfall forecast data. Fig. 2 illustrates the flood forecasting workflow using LSTM.

2.3. *Performance metrics used for machine learning modeling evaluation*

Several performance measures are utilized in this research to assess LSTM and GRU performance. They are Mean Square Error (MSE), Mean Absolute Error (MAE), Mean absolute scaled error (MASE), and Huber Loss.

MSE (Equation (1)) is the average square of the difference between

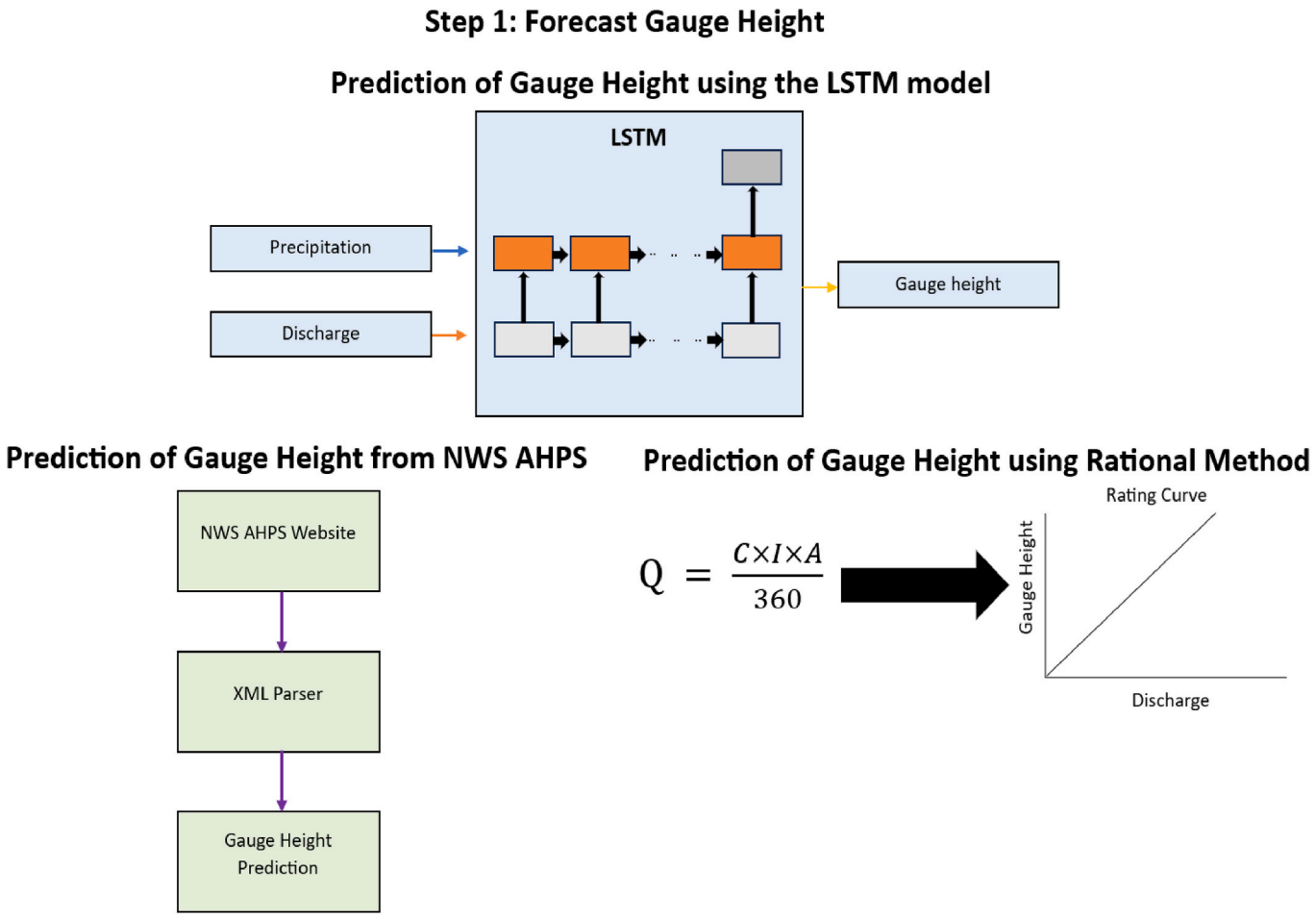


Fig. 2. Flood forecasting workflow using LSTM as the best model. NWS AHPS stands for NWS Advanced Hydrologic Prediction Service (AHPS).

the model's predicted data and the actual data throughout the whole dataset.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 \qquad \text{Equation (1)}$$

Where:

$\hat{Y}_i$ is predicted gauge height. Y_i is observed gauge height. n is the length of the dataset.

MAE (Equation (2)) is the average magnitude of the difference between the model's predicted and observed flood gauge height data for a collection of predictions and observations as a measure of the magnitude of errors for the entire dataset.

$$MAE = \frac{\sum_{i=1}^{n}|Y_i - \hat{Y}_i|}{n} \qquad \text{Equation (2)}$$

Where:

$\hat{Y}_i$ is predicted gauge height. Y_i denotes observed gauge height. n represents the length of the dataset.

MASE (Equation (3)) is an alternative to metrics like MAE to provide a more interpretable scale. To calculate MASE, we divide the MAE of the forecasting method with the MAE obtained when using the previous observation as the forecast for the next observation.

$$MASE = \frac{MAE_{forecast}}{MAE_{naive}} \qquad \text{Equation (3)}$$

Where:

$MAE_{forecast}$ is MAE of the forecast method. MAE_{naive} represents MAE obtained when using the previous observation as the forecast for the next observation.

The Huber loss (Equation (4)) is a robust loss function used in regression problems. It combines the properties of the MAE and the MSE. The Huber loss is quadratic for small error values (similar to MSE) and linear for large error values (similar to MAE), making it less sensitive to outliers than the MSE. The ideal value of Huber loss is zero; the closer the value is to zero, the better the model performance.

$$L_\delta(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{if } y - f(x) \le \delta \\ \delta \cdot \left(|y - f(x)| - \frac{1}{2}\delta^2 \right) & \text{otherwise} \end{cases} \qquad \text{Equation (4)}$$

Where:

y is the observed gauge height. $f(x)$ denotes the predicted gauge height. δ represents a threshold value.

2.4. Rational method

The Rational method (Equation (5)) is a deterministic hydrological approach commonly used for estimating peak flow rate or discharge in an ungauged watershed. It is based on the principle that the peak flow rate is directly proportional to the rainfall intensity, the area of the catchment, and a runoff coefficient that considers the characteristics of the land use and soil type in the area. This method uses a simple mathematical equation to estimate flood peak discharge (Q) based on

three inputs: the rainfall intensity (I), the drainage area (A), and the runoff coefficient (C). The equation is given as:

$$Q = \frac{C \times I \times A}{360} \quad \text{Equation (5)}$$

Equation (5) involves: The measurement of I in inches per hour (in/hr), the expression of A in acres, and the utilization of C as a dimensionless factor.

To obtain the value of Q in cubic feet per second (cfs), it is necessary to divide the product of C, I, and A by 360. The product of the values of C, I, and A is first divided by 12 to convert the unit of measurement from inches to feet and subsequently divided by 60 to convert the unit of measurement from hours to minutes. This calculation yields a factor of 1/720 ($\frac{1}{12} \times \frac{1}{60} = \frac{1}{720}$). This value is subsequently multiplied by 3600, which serves to convert minutes to seconds. The computation of Q in cfs is obtained through division by 360.

The initial step in computing the runoff coefficient involves obtaining the land use data for a specific latitude and longitude through the utilization of an application programming interface (API). A coefficient is assigned to each type of land use. The OpenWeatherMap API is utilized to obtain data on rainfall intensity. The drainage area is obtained by utilizing a digital elevation model (DEM) specific to the low-country region of SC. Initially, the metadata of DEM was extracted, encompassing details such as the pixel dimensions and transformation data. Subsequently, the provided latitude and longitude values were transformed into pixel coordinates utilizing the available transformation data. A threshold value was subsequently employed on the DEM to generate a binary mask that denotes the watershed region. The predicted gauge height is considered as a threshold value for HAND. Finally, the computation of the watershed's drainage area involves the summation of the mask, which is then multiplied by the pixel area. The calculated drainage area is expressed in units of square meters and converted to acres through division by 4047. By utilizing these three variables, it is possible to derive the maximum flood peak rate in a catchment.

A Rating curve approach is then employed to convert the maximum flood peak rate obtained from the Rational method's equation into gauge height. The Rating curve is established by the USGS as an empirical correlation linking the stage of a river to its stream discharge. The rating curve represents the correlation between the height of a measuring instrument and the volume of water flowing in a stream. The Rational method is employed in cases where there is insufficient data to facilitate flood gauge height prediction.

2.5. HAND model

We developed the HAND model as an inundation mapping approach (Nobre et al., 2011) in Python to depict the potential extent of flooding. This terrain analysis technique estimates the elevation of a point above the nearest stream or river. The model is commonly used to ascertain flood risk, drainage patterns, and erosion potential. This model is founded on the principle that water flows downhill, accumulating in low-gradient areas prone to ponding (see Nobre et al., 2011). Consequently, the vertical distance between a given point and the closest stream or river determines the likelihood of water moving downstream.

To generate the HAND model, we first use a DEM model to produce a flow accumulation map, which identifies the cells contributing to the flow of each DEM cell, typically concentrated near streams and rivers. A distance transform algorithm then determines the distance between each DEM cell to the closest stream or river. Subtracting a cell's elevation from this distance produces its HAND value. These values are used to create a HAND map, which shows the relative elevation of points above nearby streams or rivers. The HAND model employs two methods on a DEM to normalize the terrain in relation to the hydrological network. The first step computes a hydrologically corrected DEM that defines water flow paths and drainage channels. The second uses indigenous drain orientations and the drainage system to generate the nearest drainage network. This subsequently guides the HAND operator in normalizing the topology of the terrain model. The HAND model is classified into five classes based on flood depth and the severity of inundation: class 1 (0-0.5 m), class 2 (0.51-1 m), class 3 (1.1-1.5 m), class 4 (1.51-2.0 m), class 5 (>2.0 m). Flooding is predicted when water levels exceed the elevation above the nearest stream or drainage (see Nobre et al., 2011). The HAND methodology assigns a value to each pixel in a raster, which represents the relative elevation in meters between the pixel and the nearest water stream. Equation (6) provides the map algebra formula for calculating inundation that is equal to or less than the HAND value.

$$\text{HAND raster} < x \quad \text{Equation (6)}$$

where x is the gauge height value.

Fig. 3 illustrates HAND calculations, beginning with filling the DEM to remove any sinks or pits (Step 1), generating a flow direction raster (Step 2), calculating flow accumulation (Step 3), and creating a stream raster (Step 4) to estimate the amount of flooding at drainage outlets. Next, DINF (D-Infinite) is calculated (Step 5) to create a raster of flow direction from each cell to its downslope neighbor or neighbors (see Tarboton, 1997). The flow distance raster is then generated (Step 6) using the flow direction raster and the vertical distance (elevation differences). The flow distance raster is subtracted from the original DEM to get the HAND (step 7). For the last step (i.e., Step 8), Equation (6) is used to calculate the HAND-based inundation area. In this example, integer 2 is considered the flood depth.

Initially, HAND elevation data from Y. Liu et al. (2020) was used to generate a hydrological terrain raster, known as HAND, for a Hydrologic Unit Code 6 (HUC6) region in the contiguous United States (CONUS). This was achieved by utilizing a DEM with a 10-m resolution obtained from the USGS 3-D Elevation Program (3DEP) and the National Hydrography Dataset (NHD) Plus hydrography dataset. Then, the NHD data was amalgamated with terrain information to extract the hydraulic properties of the river reach and generate an all-encompassing dataset. Subsequently, Python's map algebra was employed to compute the HAND value for each raster grid, and the HAND map was categorized into several classes to reflect flood depth, as explained above.

2.6. Social media text mining

Human input is a crucial component of HAC architecture. Collecting data about how humans respond to a flooding event is often lengthy and time-consuming; however, with new technological advancements, gathering this human data has become less complicated. This research collected data generated by humans using social media, here X. We used the X API to collect human data including real-time updates, posts/tweets, and contextual information. The X API is a valuable tool for collecting X posts related to flooding because the posts provide near real-time updates on flood conditions and evacuation efforts. The X API enables developers to search for messages using particular keywords or hashtags, making it simple to collect relevant data. The X API also provides metadata about messages, such as location and time, which can be used to filter and analyze the collected data. In this study, only X posts from SC were retrieved.

In addition, we designed a text classification model to filter only those X posts that were deemed relevant to flooding issues. Specifically, we used Google's Bidirectional Encoder Representations from Transformers (BERT) package to classify X posts. BERT is a cutting-edge, pre-trained NLP model with sophisticated neural network architecture and capacity for contextual text analysis (Khan et al., 2023). The BERT model can generate high-quality representations of natural language text by simultaneously considering the entire input sequence of words to the left and right of the target word, thereby enabling more contextually relevant representations. In contrast to previous NLP models, which only consider the context of the target term's left and right word, this model

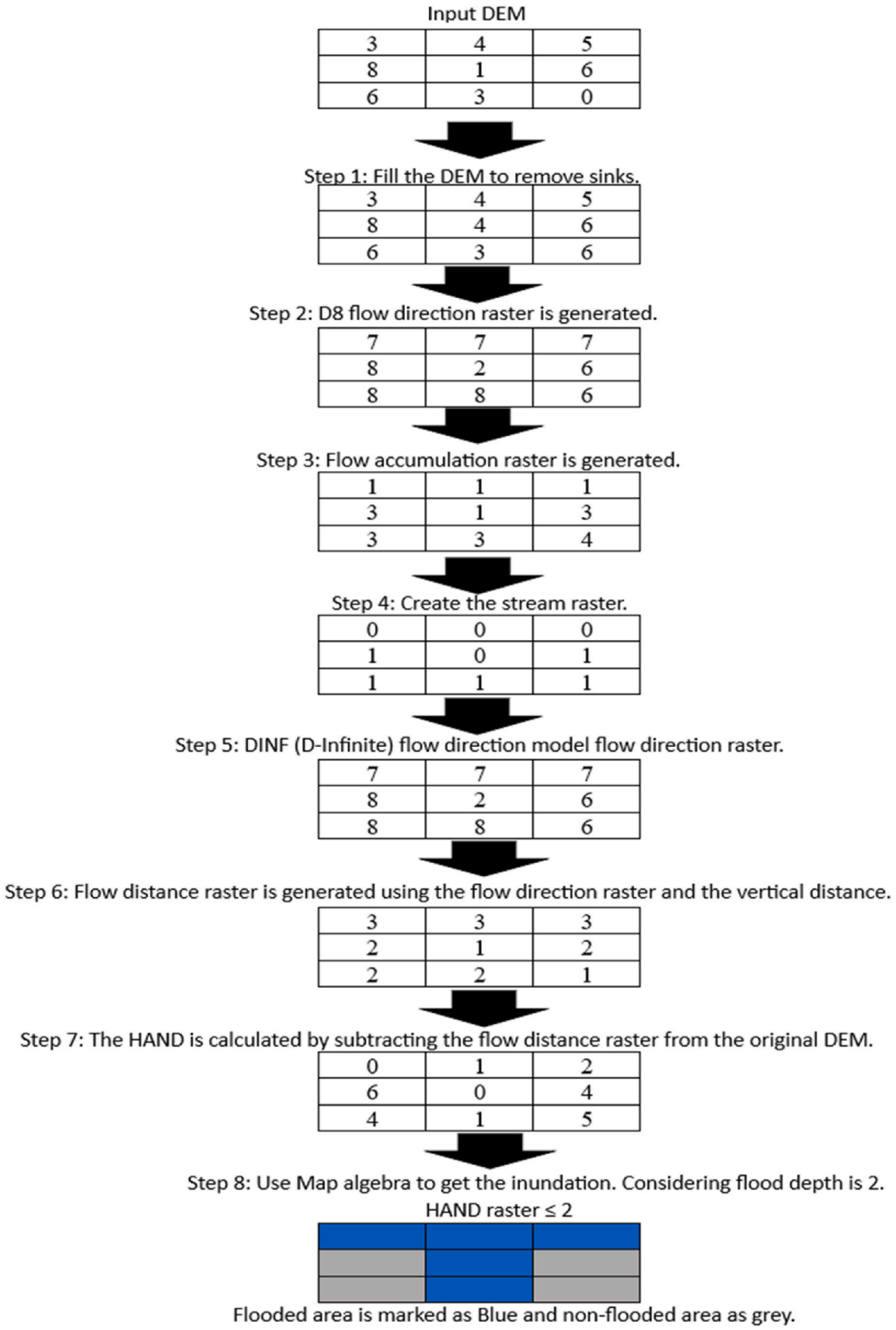


Fig. 3. The workflow of HAND model. Those numbers in a matrix are the elevation to represent a terrain in a DEM.

considers the entire sentence.

The BERT model was trained on a labeled dataset of X posts, where each post was manually annotated as either relevant or irrelevant to flooding. These X posts were gathered with keywords: “floods,” “flood emergency,” “road damage,” and “evacuation” from around the world. The labeling process involved multiple annotators to ensure consistency, and validation was performed by comparing the classifier's predictions against a held-out test set to measure accuracy and reliability. A text classifier was then built on top of BERT to automatically filter newly collected posts. Once X posts were gathered via the API, the classifier determined which posts were relevant to flood events. Fig. 4 illustrates the workflow combining BERT with the HAND model. Relevant posts identified by BERT were then integrated into the HAND model to validate inundation areas. Various geospatial datasets were used to combine HAND and BERT outputs with transportation data, enabling an assessment of flood impacts.

2.7. Performance metrics used in X post classification

To evaluate the performance of the BERT model for classifying X posts, we used three standard performance metrics: accuracy, precision, and recall. Accuracy measures the model's overall performance and represents the percentage of X posts correctly classified as relevant or irrelevant to flooding. This metric is essential for evaluating the general effectiveness of the model. Precision measures the proportion of correctly classified relevant X posts among all X posts classified as relevant by the model. This metric is essential for evaluating the accuracy of the positive predictions made by the model. The recall metric measures the proportion of correctly classified relevant X posts among all relevant X posts in the dataset. This metric is essential for evaluating the completeness of the positive predictions made by the model. Equations (7)–(9) are the accuracy, precision, and recall formulas, respectively. In these equations, *TP* denotes true positive, *TN* is true negative, *FP* represents false positive, FN represents false negative, *P* is total positive classes, and *N* denotes total negative classes. In addition, $P + N$ represents the total number of X posts, where *P* corresponds to posts that are flood-related, and *N* corresponds to posts that are not flood-related.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{P} + \text{N}} \qquad \text{(Equation 7)}$$

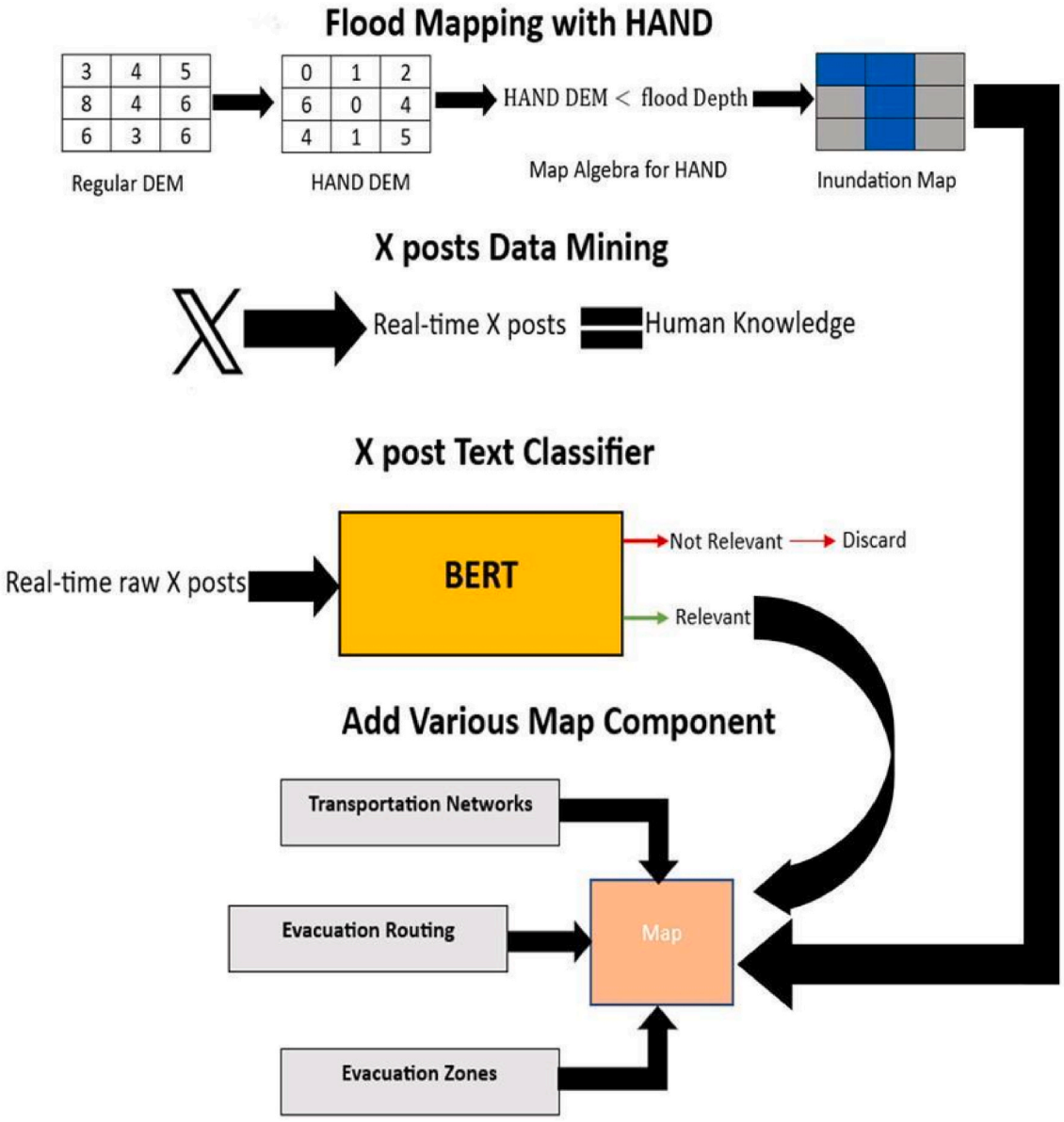


Fig. 4. The workflows of the system after LSTM prediction.

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{(Equation 8)}$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad \text{(Equation 9)}$$

2.8. Evacuation Re-routing

A routing algorithm was incorporated into the HAC system to recommend alternative evacuation routes in the event of flooding. Specifically, the Leaflet Routing Machine, accessed via the GraphHopper API, was used to generate re-routing options during a major flooding event on January 10, 2024, in Lowcountry, SC. The GraphHopper API was integrated into the prototype to compute the shortest paths as well as alternative routes between multiple locations. Key parameters passed to the API included the algorithm type (alternative_route), maximum number of routes (max_paths), maximum weight factor (max_weight_factor), and maximum sharing factor (max_share_factor). The primary parameters employed for our case studies are summarized below:

(i) Algorithm Type (alternative_route): This parameter specifies the algorithm used for routing, with alternative_route being particularly useful for evacuation as it provides several route options.
(ii) Maximum Number of Routes (max_paths): Determines the number of alternative routes to generate. In evacuation scenarios, having multiple paths ensures that there are options available if the primary route becomes impassable.
(iii) Maximum Weight Factor (max_weight_factor): This parameter influences the maximum weight of the alternative paths, which can be interpreted as a measure of route efficiency in terms of distance.
(iv) Maximum Sharing Factor (max_share_factor): This parameter controls the degree of similarity between the alternative routes. A lower sharing factor tells the algorithm to provide routes that diverge from each other, which can increase the chances of avoiding blocked areas.

These parameters ensure that multiple evacuation routes are generated and that flooded areas are avoided as much as possible. After the API returned the routes, each route was checked to see if it was in the inundation area or not. The fastest route, which was out of the inundation area, was then selected and displayed on the map to guide

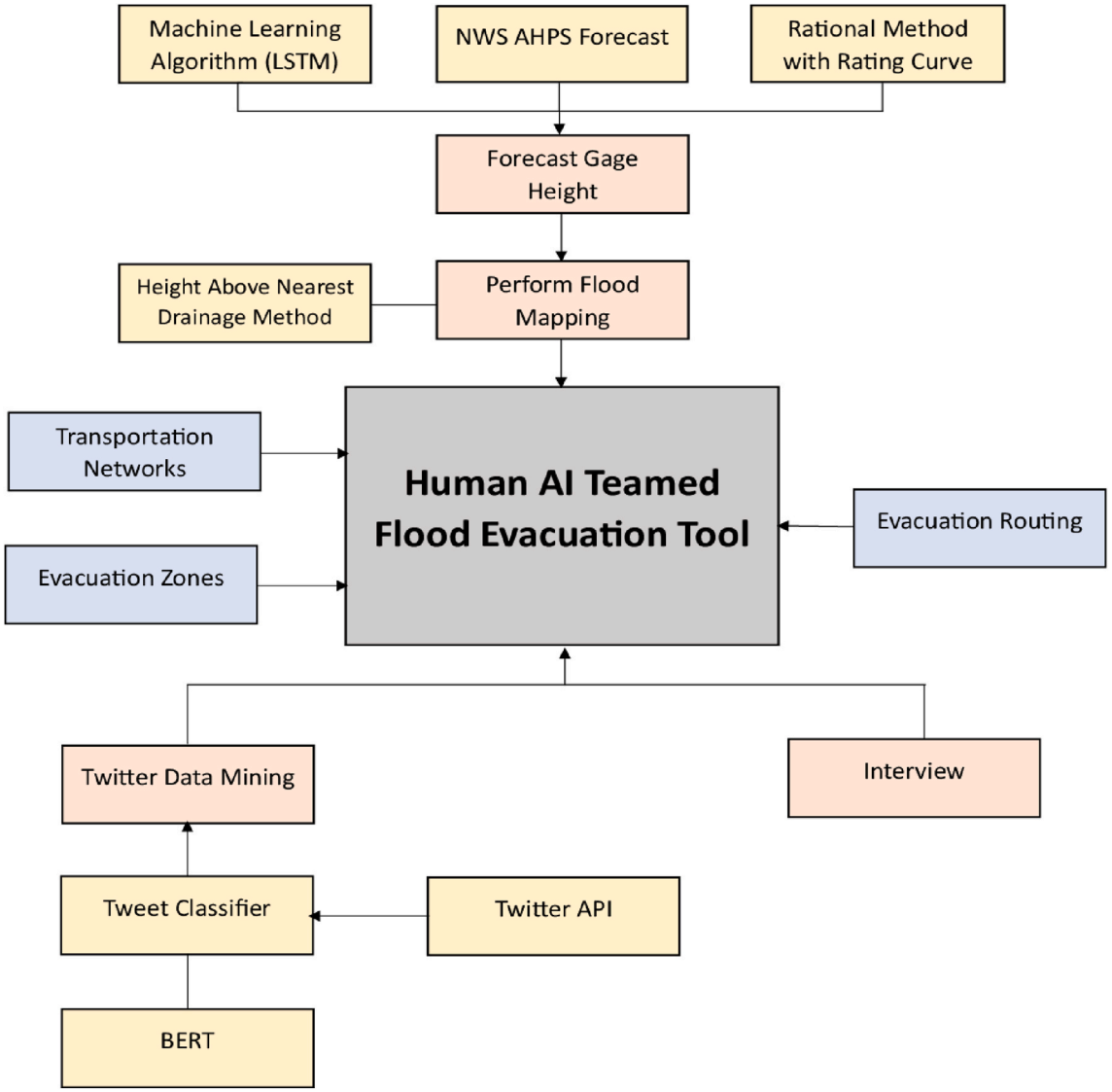


Fig. 5. The overall workflow of the HAC flood evacuation system.

citizens toward a safe evacuation.

2.9. HAC system structure and workflow

Fig. 5 illustrates the workflow of the HAC system. The system combines multiple modules including a machine learning prediction model, Rational method, river hydraulic model, BERT text mining approach, evacuation re-routing model, and visualization. The system first predicts gauge height using machine learning approaches and uses the Rational method if USGS gauging data is not available for a particular watershed. The estimated gauge height or flood depth is then used in the HAND model for flood inundation mapping. The inundation outcome is integrated into the BERT model and X posts to validate the inundation areas. Finally, the system uses the Graphhopper API to avoid inundated roads and suggest rerouting. To perform evacuation re-routing, a Leaflet routing machine, and a JavaScript library for interactive re-routing in web applications were used to connect with the Graphhopper API. The Graphhopper API provides various re-routing algorithms using the 'alternative_route' algorithm, which generates multiple alternative routes for a given start and end point.

3. Results and applications

3.1. Flood gauge height forecasting

A total of 180 models were trained for both LSTM and GRU models using the Optuna algorithm, from which the top three models were selected. Optuna is an automatic hyperparameter optimization algorithm, particularly designed to dynamically construct the search spaces for hyperparameter optimization. To tune the hyperparameters, we minimized the validation loss function in Optuna. In each gauging station, Optuna computed the number of neurons in each layer as well as dropout rate, and the number of epochs. The number of neurons varied significantly among hidden layers with the least number in the 6th layer (5 to 15) and the maximum value presented in the first hidden layer (100 to 200). The drop-out rate ranged between 0.1 and 0.5 with the Epoch number of 50-200. The three gauging stations that were used include Turkey Creek (USGS02172035), South Fork Edisto River (USGS02173000), and North Fork Edisto River (USGS02173500). We used 03/01/2013 to 05/08/2023 datasets to simulate gauge height values for USGS02173500 and USGS0217035. Due to data unavailability, we used 01/01/2020 to 05/08/2023 period to predict and forecast gauge height values at USGS02173000. Missing values in the gauge height data were addressed using forward fill interpolation, where each missing value was replaced with the most recent preceding observation to maintain temporal continuity.

The performance of the LSTM and GRU models exhibited a high degree of similarity. However, the LSTM model exhibited slightly superior performance, particularly on the test dataset. Therefore, only the LSTM model is presented here. LSTM was particularly successful in capturing flood peak rates and time to peak, which are two important factors for flood emergency decisions. Table 1 shows the performance achieved by LSTM as the best model for all gauging stations in testing, validation, and training periods, respectively. The LSTM was proficient in simulating gauge heights across the three gauging stations, with respect to the multiple performance metrics. Error estimation metrics such as Huber Loss, MSE, and MAE were comparably low across different gauging stations.

Table 1
LSTM performance across three gauging stations.

Station	MASE	Huber Loss	MSE	MAE
Training Period				
USGS02173500	0.0028	0.0025	0.0050	0.0430
USGS02173000	0.0063	0.0020	0.0040	0.0447
USGS0217035	0.0094	0.0188	0.0890	0.0066
Testing Period				
USGS02173500	0.0019	0.0120	0.02404	0.1284
USGS02173000	0.0066	0.0248	0.0498	0.1619
USGS0217035	0.00460	0.0177	0.0354	0.1406
Validation Period				
USGS02173500	0.0027	0.0070	0.0140	0.0887
USGS02173000	0.0052	0.0075	0.0150	0.0634
USGS0217035	0.0078	0.0065	0.0131	0.0696

Prediction results of LSTM are visualized in Figs. 6–8. As shown, the LSTM was able to accurately predict a quick rise and fall of the flood gauge height values, particularly at USGS02173000; even though the slope and behavior of the gauge height data are not well captured in the Turkey Creek gauging station (USGS0217035). In addition, low gauge height values are not well captured by LSTM across all three gauging stations. Specifically, when the gauge height values were less than 4 m, the LSTM performance dropped significantly. LSTM appears to be sensitive to the widespread scale of data, so the model might learn that the low gauge height values carry no information. In addition, low gauge height values, or values near zero, contribute little additional information to the learning algorithms, as they typically correspond to normal or base-flow conditions rather than extreme events. Including these low values could potentially bias the model toward predicting non-flood conditions and reduce its sensitivity to significant flood events. Therefore, our analysis focused primarily on higher gauge values that are most relevant for flood prediction. These higher values reflect exceedance runoff, which represents the additional water volume entering the river system during intense precipitation or storm events and directly contributes to rapid river level increases. By concentrating on these elevated values, the model is better able to learn patterns associated with flood dynamics and improve prediction accuracy for extreme events.

There are cell states in the LSTM networks that can be interpreted as storage capacity often used in traditional flood modeling schemes. In LSTM, the updating of internal cell states (or storages) is regulated through a number of gates: the first gate regulates the storage depletion, the second one regulates storage fluctuations, and the third gate regulates the storage's outflow. The elaborate gated design of the LSTM partly solves the long-term dependency problem in gauge height prediction, although, the structure of LSTMs is designed in a sequential manner that cannot directly connect two nonadjacent portions (positions) of a time series. Further, LSTM can suffer from vanishing or exploding gradient problem during training. The vanishing gradient problem can happen in LSTMs when the gradients are multiplied repeatedly during backpropagation through the recurrent connections. The vanishing or exploding gradients issues can reveal itself in an error signal during the backward pass of the network training that either grows against infinity or diminishes towards zero. When the gradients become too small, it is hard for the model to learn long-term dependencies in the dataset, resulting in unstable training.

One can also note the insignificant differences between modeling performances of USGS02173000 and USGS02173500. These two gauging stations are part of a large Edisto River Basin. This concludes that LSTM was able to learn the gauge height fluctuations and dependencies across a large basin. However, the South Fork Edisto River (USGS02173000) showed more agreement with LSTM predictions when the gauge height varies between 5 and 6.5 ft. Unlike those gauging stations, the LSTM provided less sophisticated predictions for USGS0217035 flood gauge height prediction. USGS0217035 gauging station is in a forest-dominated watershed. Perhaps other physical parameters such as land use cover, infiltration rate, time of concentration are important in flood generation mechanisms in which LSTM was unable to capture these dynamics.

3.2. Social media data analysis

In this research, we leveraged unstructured social media data to enrich the human dimension of the HAC system. The real-time nature of social media, especially X, was instrumental in adding real-time human

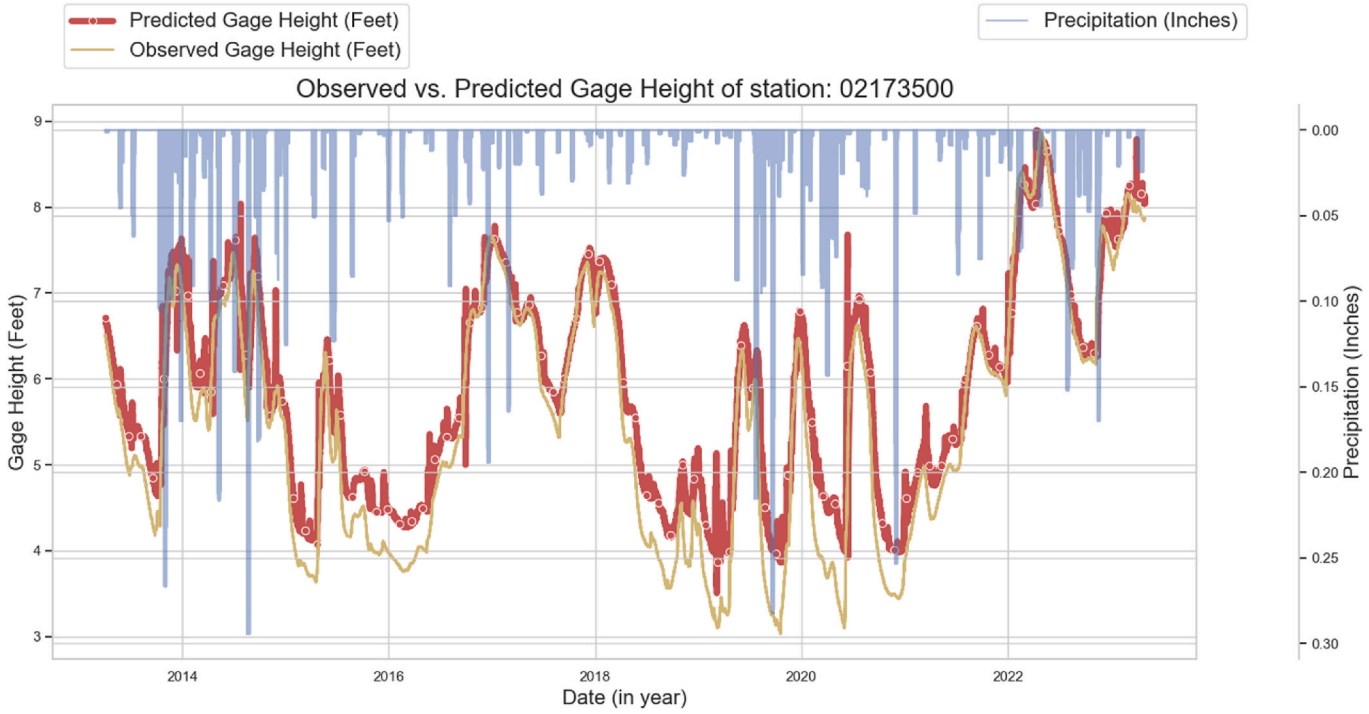


Fig. 6. USGS02173500 flood gauge height prediction for training, testing and validation periods (03/01/2013 to 05/08/2023).

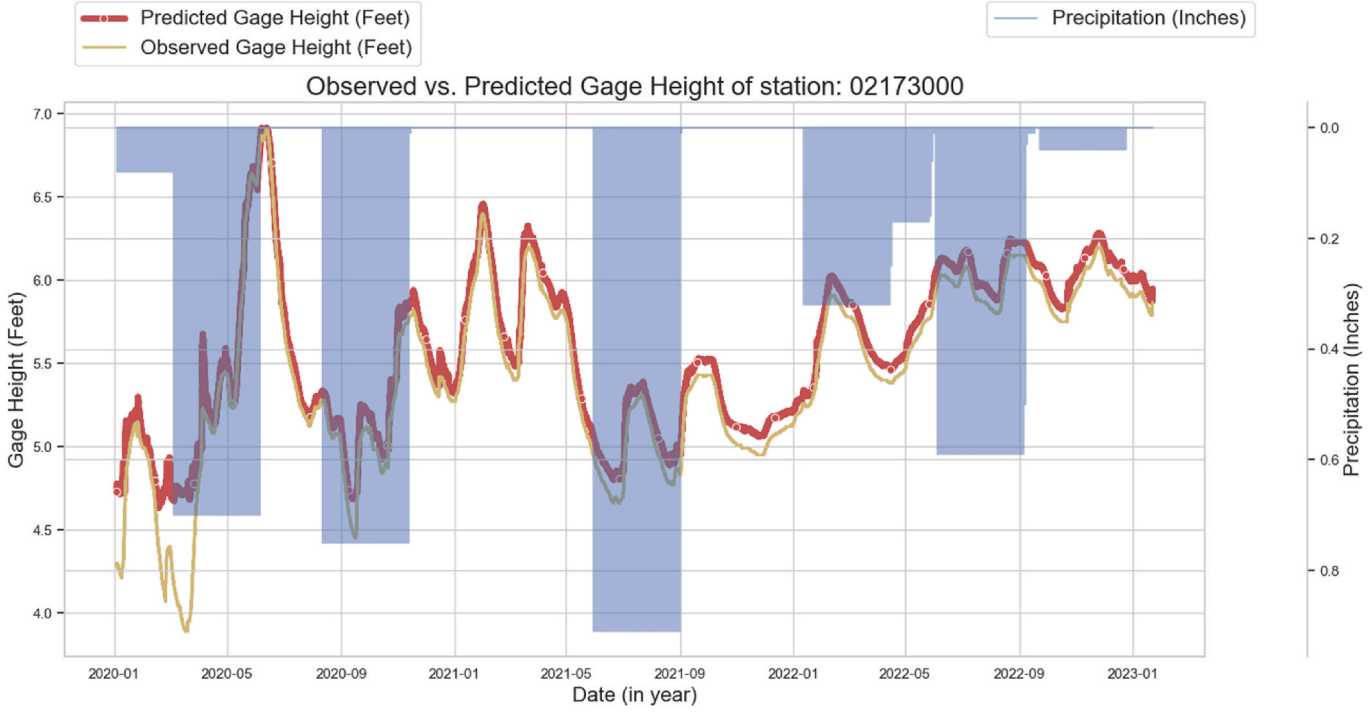


Fig. 7. USGS02173000 flood gauge height prediction for training, testing and validation periods (01/01/2020 to 05/08/2023).

knowledge to the HAC system. The social media text data provided immediacy and diversity of perspectives, offering contextual richness to incorporating human data into the system. We used the X API to collect and mine text data. The keywords that were used to search for X posts include "floods," "flood emergency," "road damage," and "evacuation". These keywords can be customized by the user to collect data on specific flood events or locations.

We filtered the X text dataset and removed entries that were not relevant to flooding. The filtration was performed via an X post-classifier model that was constructed utilizing an NLP model called BERT algorithm. We first created a text classifier on top of the BERT model. We then collected the dataset from various sources such as WALLACH (2018), Preda (2020), Stepanenko and Liubko (2020), Alam et al. (2021), and Suresh (2021) to train BERT. Overall, a dataset of approximately 60,000 X text posts was gathered and manually annotated to indicate whether each post was related to flooding or not. This dataset

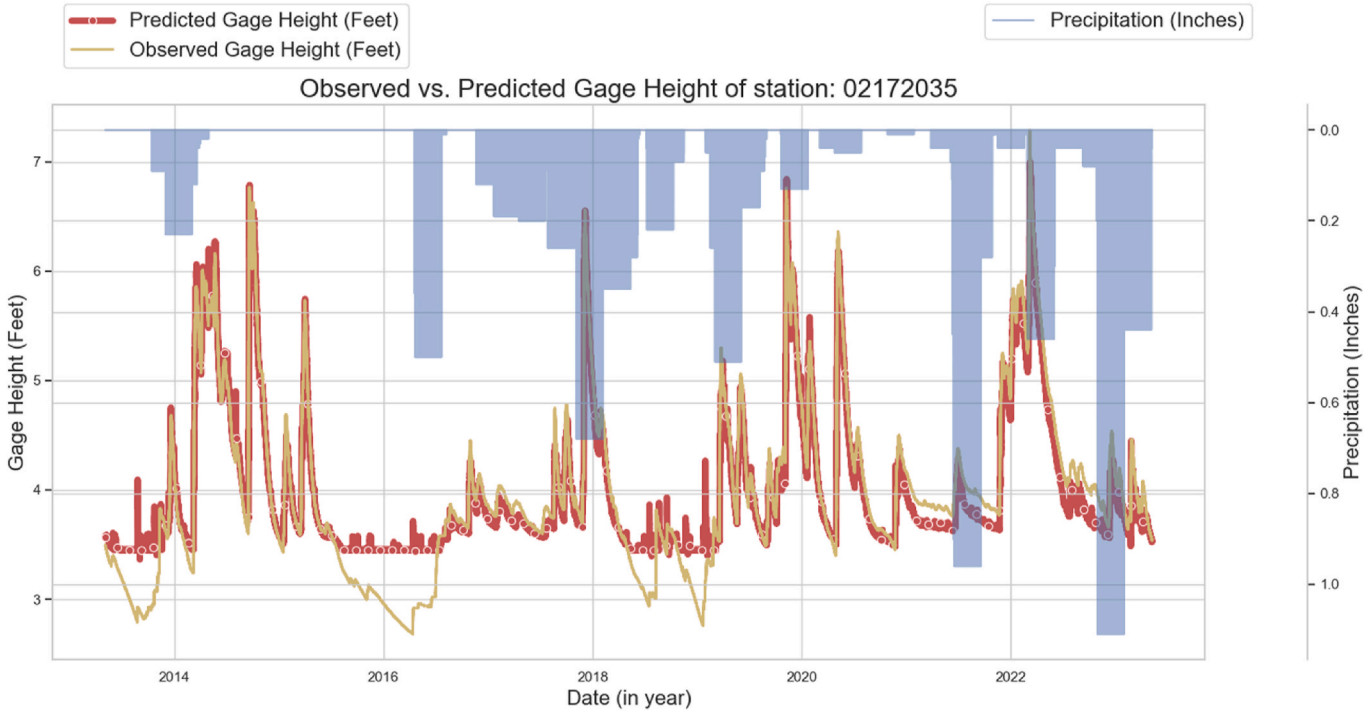


Fig. 8. USGS0217035 flood gauge height prediction for training, testing and validation periods (03/01/2013 to 05/08/2023).

was used to develop an X text post classifier model. The text data was partitioned into two datasets, namely a training set and a testing set, with a ratio of 75%:25%. These data sources also contain irrelevant X posts so that BERT could learn to distinguish between relevant and irrelevant X posts by accumulating irrelevant X posts alongside relevant ones during training. By including irrelevant X posts in the training data, the model learned to differentiate between various categories of X posts and identified which specific features or keywords indicate whether an X post is relevant or irrelevant to flooding. Each text was then given a category of 0 (not relevant) or 1 (relevant). During the process of fine-tuning, the BERT model learned to identify key flood-related textual features and used them to make accurate predictions.

The fine-tuning process involved training the BERT model on the labeled dataset and adjusting its parameters to classify relevant X posts. A backpropagation method was then used for the text fine-tuning process. The model was trained for 30 epochs. The model's predictions were compared with the true labels, and the model's parameters were adjusted to minimize the difference between the predictions and the true labels. Once the BERT model was fine-tuned, it was then used to classify new, unlabeled X posts as either relevant or irrelevant to flooding. The model's output is a probability score indicating the likelihood that the X post is relevant to flooding. If the probability score is above a threshold of 80%, the X post is classified as relevant to flooding. After the X posts were collected using X API in real-time, we performed text classification of collected X posts. This text classification decided whether the X post was relevant to a flood disaster or not.

Fig. 9 shows the BERT architecture designed to classify the X posts. The input dimension, corresponding to 60,000 text samples, was specified as None in order to provide flexibility during model construction. By setting the dimension to None, the framework automatically adapts to any scalar size rather than fixing it to a predefined value. This approach allows the model to generalize across varying input lengths while maintaining compatibility with the computational graph and training process (Abadi et al., 2016). So, the input dimension was arbitrary to the input text length. A pre-processed layer obtained from a pre-existing saved text preprocessing layer was then utilized to preprocess the text data in TensorFlow Hub. This layer served as a companion to the BERT model, facilitating the preprocessing of plain text inputs into the specific format that BERT required. The pre-processed layer's output was linked to the input of the BERT encoder layer sourced from a pre-existing TensorFlow Hub model that was trained beforehand.

BERT utilized Transformer architecture and a deep, pre-trained neural network to generate dense vector representations for natural language. The BERT model employed 12 hidden layers, also known as Transformer blocks, with a hidden size of 768 and 12 attention heads. The weights utilized in this model correspond to those disclosed by the primary authors of BERT. The outputs of the encoder consist of two components: the “pooled_output,” which served to encapsulate the entirety of the input sequence, and the “sequence_output,” which represented each individual token within the context of the sequence. The output obtained from “pooled_output,” was linked to the dropout layer with a rate of 0.1. The dropout was subsequently linked to a densely connected output layer.

The BERT model attained an accuracy rate of 88.5% along with a precision rate of 84% and a recall rate of 85% during the training period. Similarly, during the testing phase, the model achieved an accuracy rate of 89%, a precision rate of 81%, and a recall rate of 94%. The predicted versus actual values of each class were obtained using a confusion matrix (see Table 2). As shown, the number of positive values predicted by BERT are much higher than negative values.

BERT's architecture, especially its bidirectional mechanism, was pivotal in understanding the context behind text classification. When we examined BERT for real time X post classification, its performance highlighted the model's ability to effectively utilize human-generated data in the context of evolving flood situations. It is interesting to note that some X posts can be false or disinformation. However, we checked the posts with other reliable information such as local news, NWS posts, etc. and we found that the percentage of the false X posts was insignificant. BERT demonstrated its efficacy as a vital tool for contemporary flood prediction systems by efficiently eliminating extraneous data and focusing on relevant flood-related information. Although, the progress of X post mining was considerably affected by the X decision not to support free access to the API.

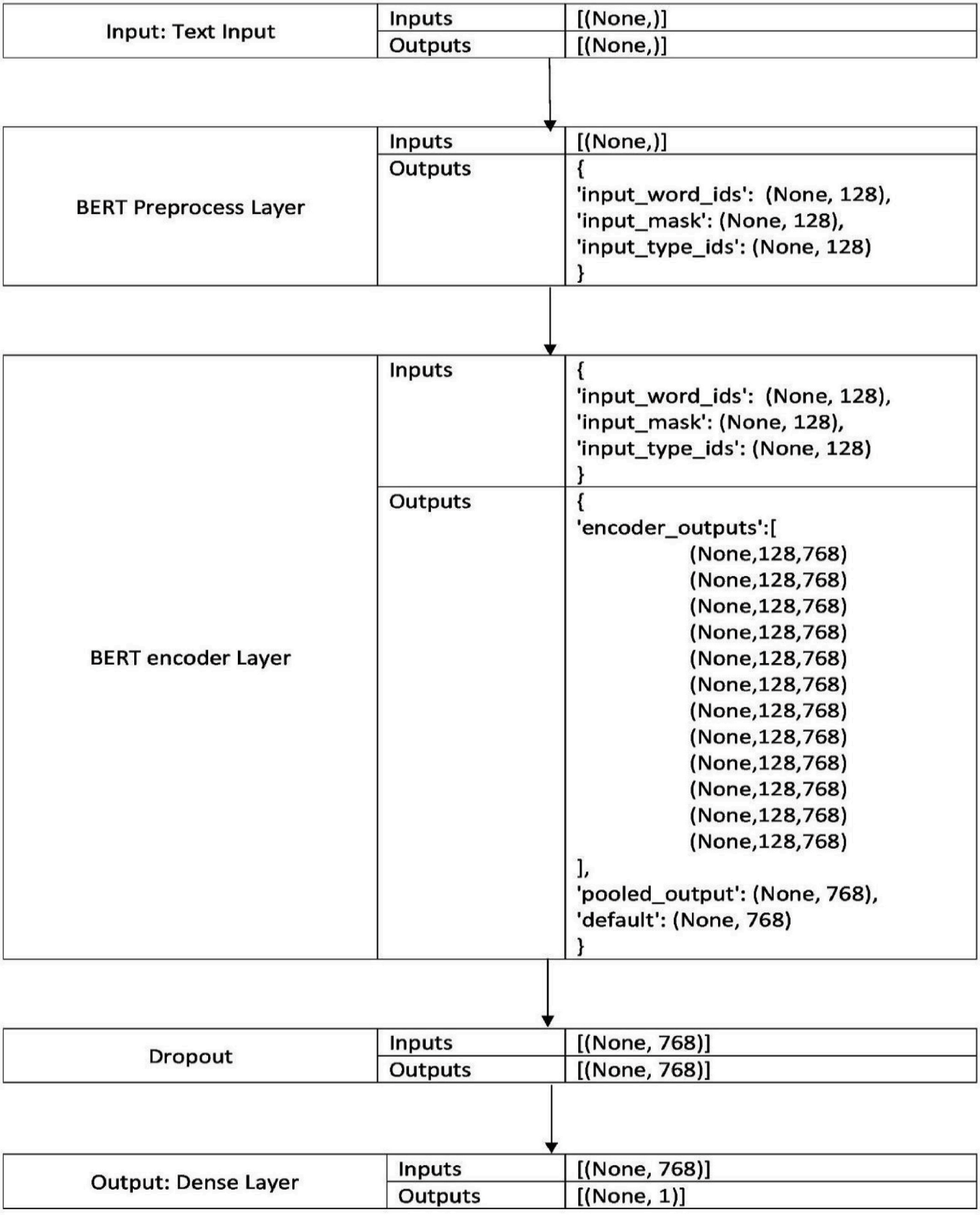


Fig. 9. Neural network architecture of X post classifier constructed in this research.

Table 2
Confusion Matrix for the test set of the BERT model.

		Predicted Values	
		Negative	Positive
Actual Value	Negative	8183	1238
	Positive	319	5209

3.3. Evacuation Re-routing results

The GraphHopper API was employed to facilitate evacuation re-routing. Using the alternative_route parameter, the API generated multiple potential routes, from which the best route that avoided inundated areas was selected as the alternative. When multiple routes were returned, each was manually verified to ensure it remained clear of flooded regions. The fastest safe route between points A and B was then chosen (see Fig. 10) and displayed on the map, providing a reliable guide for safe evacuation. The system was able to display the optimal route within seconds, actively avoiding inundated areas to maximize the safety of evacuation paths.

4. Conclusions

The HAC system developed in this study as a flood evacuation tool demonstrated high efficacy in forecasting river gauge heights and recommending evacuation rerouting by integrating LSTM models with a river hydraulic model and social media data. Incorporating real-time human information from social media added a critical human dimension, enabling the identification of regions requiring prompt aid and evacuation. The prototype showed considerable potential for enhancing disaster response and evacuation strategies in low-lying areas of SC, with applicability to other flood-prone regions. The high accuracy and precision of the LSTM and BERT models highlight the effectiveness of machine learning and natural language processing in predicting river gauge heights and filtering relevant social media data to inform timely decision-making.

Traditional machine learning models typically rely on historical data for predictions, which can be insufficient in dynamic and unpredictable scenarios such as flooding, where real-time information is crucial. While the standalone LSTM achieved strong numerical accuracy, it lacked real-time correction from human inputs, resulting in slower adaptation to localized flooding. To address this limitation, we developed a methodology that integrates machine learning predictions with real-time social media data (X) within a geographic interface. This cartographic representation serves as a bridge between machine intelligence and human inputs, enabling mutual reinforcement and improving prediction accuracy. By combining these two information sources through a visualization map, the approach provides a cohesive and actionable perspective on the HAC based evacuation decision.

The HAC system represents a novel approach to human-AI



Fig. 10. Map showing the re-routed path along Hwy 41 (near Turkey Creek River; USGS02172035) during the January 10, 2024, flooding event in Lowcountry, SC. The interactive mapping and geospatial analysis were performed using the Leafmap Python package (Wu, 2021).

collaboration in flood evacuation, strategically integrating social media data, machine learning, and flood inundation models to analyze dynamic scenarios and recommend optimized evacuation routes. Tested in real time in the flood-prone Lowcountry of SC, the system demonstrated significant practical potential for improving emergency response. By augmenting human decision-making with AI-driven insights, HAC enhances the robustness, accuracy, and effectiveness of real-time evacuation strategies. As societal demands for timely and precise flood evacuation decisions increase, advanced technologies like HAC are poised to play an increasingly critical role. Traditional flood management methods are gradually being complemented—and in some cases replaced—by sophisticated computational approaches, underscoring the importance of human-AI collaboration in high-stakes, real-time decision-making.

Future research should focus on incorporating additional human-derived data sources, such as crowd-sourced reports, IoT sensor data, and mobile location information, to further refine the HAC system and strengthen its reliability in diverse contexts. Key challenges remain regarding workflow performance, predictive uncertainties, and the practical impact of social media data, all of which will require systematic evaluation in real-world settings. Addressing these questions will be important for establishing trust and ensuring that HAC systems can function as dependable partners in emergency management. While this study represents an early proof of concept, we anticipate that as HAC approaches mature, the current human-only process will evolve into a more transparent and adaptive human-AI teaming process—one that is capable of delivering faster, more robust, and more context-aware evacuation decisions. By laying this foundation, the present study opens new avenues for technological innovation, policy development, and operational improvements in flood management practices worldwide.

CRediT authorship contribution statement

Rishav Karanjit: Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Vidya Samadi:** Writing – review & editing, Validation, Supervision, Software, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Amanda L. Hughes:** Writing – review & editing, Validation, Supervision, Methodology, Funding acquisition. **Pamela Murray-Tuite:** Writing – review & editing, Validation, Supervision, Methodology, Funding acquisition. **Keri K. Stephens:** Writing – review & editing, Supervision, Funding acquisition, Formal analysis.

Software availability

4.1. Name of packages

Human-AI Convergence (HAC).

4.2. Developers

Rishav Karanjit, Vidya Samadi, Amanda L. Hughes, Pamela Murray-Tuite, Keri K. Stephens.

4.3. Date first available

Jan. 10, 2025.

4.4. Package availability

The package will be available after publication.

4.5. License

MIT License.

4.6. Software requirements

OS, numpy, re, tensorflow, torch, geopy. geocoders, plotly, hydrofunctions, sklearn, pennylane, matplotlib, and CUDA-enabled PyTorch for GPU acceleration.

Declaration of competing interest

There is no conflict of interest between the authors.

Acknowledgements

This research is supported by the US National Science Foundation (NSF) Directorate of Engineering (Grant # CMMI 2125283). Clemson University (USA) is acknowledged for generous allotment of computing time on the Palmetto cluster. We acknowledge the NSF funded the Advanced Cyberinfrastructure Coordination Ecosystem: Services and Support (ACCESS) program for providing free cloud credits to deploy HAC system. The HAC system is available at http://floodevacuationtool.clemson.edu/. The source code and data are available upon request.

Appendix A

Pseudocode

Pseudocode 1: Calculate HAND from DEM

```
requires: flood_depth, TL, BR, hand_DEM_path

hand_dataset <- OPEN_GIS_DATASET(hand_DEM_path)
band <- GET_BAND(hand_dataset, 1)

transform <- GET_DATASET_GEOTRANSFORM(hand_dataset)
inv_transform <- CALCULATE_INVERSE_GEOTRANSFORM(transform)

top_left_pixel <- APPLY_INVERSE_GEOTRANSFORM(inv_transform, TL.longitude, TL.latitude)
bottom_right_pixel <- APPLY_INVERSE_GEOTRANSFORM(inv_transform, BR.longitude, BR.latitude)

start_x <- MAX(0, INT(MIN(top_left_pixel.x, bottom_right_pixel.x)))
end_x <- MIN(GET_DATASET_WIDTH(hand_dataset), INT(MAX(top_left_pixel.x, bottom_right_pixel.x)) + 1)
start_y <- MAX(0, INT(MIN(top_left_pixel.y, bottom_right_pixel.y)))
end_y <- MIN(GET_DATASET_HEIGHT(hand_dataset), INT(MAX(top_left_pixel.y, bottom_right_pixel.y)) + 1)

hand_array_clipped <- READ_RASTER_DATA_AS_ARRAY(band, start_x, start_y, width <- end_x - start_x, height <- end_y - start_y)

no_data_value <- GET_BAND_NO_DATA_VALUE(band)
IF no_data_value IS NOT NONE:
  REPLACE_ARRAY_VALUES(hand_array_clipped, no_data_value, NaN)

inundated_mask <- CREATE_MASK_WHERE(hand_array_clipped IS LESS OR EQUAL TO flood_depth AND NOT NaN)

bins <- DEFINE_BINS([0, 0.5, 1, 1.5, 2], flood_depth)

zone_indices <- ASSIGN_TO_BINS(hand_array_clipped[WHERE inundated_mask], bins)

Zone1, Zone2, Zone3, Zone4, Zone5 <- INITIALIZE_EMPTY_LISTS()
lat_list, lon_list <- INITIALIZE_EMPTY_LISTS()

FOR zone_number FROM 1 TO LENGTH(bins) + 1:
  zone_mask <- IDENTIFY_MASK_FOR_ZONE(zone_indices, zone_number)
  IF zone_mask CONTAINS TRUE VALUES:
    inundated_indices <- GET_TRUE_INDICES(inundated_mask)
    selected_indices <- FILTER_INDICES_BY_ZONE(inundated_indices, zone_mask)
    world_coords <- CONVERT_PIXELS_TO_WORLD_COORDINATES(selected_indices, transform, start_x, start_y)
    APPEND_ZONE_COORDINATES(Zone{zone_number}, world_coords)
    APPEND_LAT_LON_FROM_COORDINATES(lat_list, lon_list, world_coords)

FOR zone_number FROM LENGTH(bins) + 1 TO 5:
  CLEAR_ZONE_LIST(Zone{zone_number})

RETURN Zone1, Zone2, Zone3, Zone4, Zone5, lat_list, lon_list
```

This pseudocode1 requires flood depth (flood_depth), top left clipped rectangle coordinate (TL), bottom right clipped rectangle coordinate (BR) and HAND DEM path (hand_DEM_path). Functions used in this pseudo-code:

- OPEN_GIS_DATASET represents the process of opening the GIS file.
- GET_BAND, GET_DATASET_GEOTRANSFORM, and similar functions represent various GIS data operations.
- CALCULATE_INVERSE_GEOTRANSFORM calculates the inverse geotransformation matrix.
- APPLY_INVERSE_GEOTRANSFORM applies the inverse geotransform to convert world coordinates to pixel coordinates.
- READ_RASTER_DATA_AS_ARRAY reads raster data from the GIS file into a numeric array.
- REPLACE_ARRAY_VALUES replaces specific values in an array with another value.
- CREATE_MASK_WHERE creates a boolean mask based on a condition.
- DEFINE_BINS, ASSIGN_TO_BINS, and IDENTIFY_MASK_FOR_ZONE are used for categorizing the data into different zones based on the flood depth.
- CONVERT_PIXELS_TO_WORLD_COORDINATES converts pixel coordinates back to world.

Pseudocode 2: Forecast gauge height in real-time

requires: flood_station, period, scaler_path, model_path

```
TRY:
  herring <- NWIS_WEB_SERVICE(flood_station, 'iv', period)
  data <- EXTRACT_AND_PROCESS_DATA(herring)
CATCH Exception:
  HANDLE_DATA_RETRIEVAL_ERROR(flood_station)
  RETURN NONE

scaler <- LOAD_SCALER(scaler_path)
scaled_data <- PREPARE_DATA_FOR_PREDICTION(data, scaler)

custom_objects <- SETUP_MODEL_CUSTOM_OBJECTS()
regressor <- LOAD_MODEL(model_path, custom_objects)

prediction <- regressor.PREDICT(scaled_data)

inversed_gage_height <- POST_PROCESS_PREDICTIONS(prediction, scaler)

df_predictions <- PREPARE_PREDICTIONS_DATAFRAME(inversed_gage_height, data)

RETURN df_predictions
```

Pseudocode 2 requires identifier for the flood station (flood_station), time period over which to fetch data (period), path to the directory where the scaler files are stored (scaler_path) and path to the directory where the trained machine learning model files are stored (model_path). Function used in this pseudocode are:

- NWIS_WEB_SERVICE: Uses NWIS API to perform GET request and fetch data from it.
- EXTRACT_AND_PROCESS_DATA: Performs operations like fetching data, resampling, and timezone localization.
- HANDLE_DATA_RETRIEVAL_ERROR: Encapsulates error handling for data retrieval issues.
- LOAD_SCALER: Loads and returns scaler.
- PREPARE_DATA_FOR_PREDICTION: Prepares data scaling and preparation for input into the predictive model.
- LOAD_MODEL: Loads and return machine learning model.
- PREDICT: Predict and return predicted data.
- SETUP_MODEL_CUSTOM_OBJECTS: Represents the setup for custom objects required by the model, such as custom loss functions or metrics.
- POST_PROCESS_PREDICTIONS: Performs the post-processing of predictions to convert them from the scaled form back to the original measurement scale.
- PREPARE_PREDICTIONS_DATAFRAME: Prepares the creation of a DataFrame with predictions, including setting up the index with appropriate timestamps.

Data availability

Data will be made available on request.

References

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., others, 2016. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. arXiv preprint arXiv:1603.04467v2 1–19. https://doi.org/10.48550/arXiv.1603.04467.

Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M., 2019. Optuna: a next-generation hyperparameter optimization framework. Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2623–2631.

Alam, F., Qazi, U., Imran, M., Ofli, F., 2021. Humaid: human-annotated disaster incidents data from Twitter with deep learning benchmarks. Proceedings of the International AAAI Conference on Web and Social Media 15, 933–942.

Bansal, G., Nushi, B., Kamar, E., Lasecki, W.S., Weld, D.S., Horvitz, E., 2019. Beyond accuracy: the role of mental models in human-AI team performance. Proceedings of the AAAI conference on human computation and crowdsourcing 7, 2–11.

Bennett, G., Yang, L., Simeonova, B., 2017. A Heuristic Approach to Flood Evacuation Planning.

Cho, K.a., 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078.

Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. Neural Comput. 9, 1735–1780.

Khan, W., Daud, A., Khan, K., Muhammad, S., Haq, R., 2023. Exploring the frontiers of deep learning and natural language processing: a comprehensive overview of key challenges and emerging trends. Natural Language Processing Journal, 100026.

Liang, C., Proft, J., Andersen, E., Knepper, R.A., 2019. Implicit communication of actionable information in human-ai teams. Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, pp. 1–13.

Liu, Y., Tarboton, D.G., Maidment, D.R., 2020. Height Above Nearest Drainage (HAND) and Hydraulic Property Table for CONUS. Tech. Rep. Oak Ridge National Lab. (ORNL), Oak Ridge. Oak Ridge, TN (United States).

Madni, A.M., Madni, C.C., 2018. Architectural framework for exploring adaptive human-machine teaming options in simulated dynamic environments. Systems 6, 44.

McKinney, W., 2010. Data structures for statistical computing in python. Proceedings of the 9th Python in Science Conference 445 (1), 51–56.

McNeese, N.J., Demir, M., Cooke, N.J., Myers, C., 2018. Teaming with a synthetic teammate: insights into human-autonomy teaming. Hum. Factors 60, 262–273.

Nobre, A.D., Cuartas, L.A., Hodnett, M., Rennó, C.D., Rodrigues, G., Silveira, A., Saleska, S., 2011. Height above the Nearest Drainage–a hydrologically relevant new terrain model. J. Hydrol. 404, 13–29.

Ong, C., McGee, K., Chuah, T.L., 2012. Closing the human-AI team-mate gap: how changes to displayed information impact player behavior towards computer teammates. Proceedings of the 24th Australian Computer-Human Interaction Conference, pp. 433–439.

Phillips, P., 2020. Flooding forces closure of several Charleston-area roads. Available at: https://www.live5news.com/2020/03/05/list-flooding-forces-closure-several-downtown-roads/. (Accessed 3 January 2024).

Preda, G., 2020. COVID19 Tweets. *COVID19 Tweets*. Kaggle. https://doi.org/10.34740/KAGGLE/DSV/1451513.

Samadi, V., Stephens, K.K., Hughes, A., Murray-Tuite, P., 2024. Challenges and opportunities when bringing machines onto the team: Human-AI teaming and flood evacuation decisions. Environ. Model. Software 175, 105976.
Seeber, I., Bittner, E., Briggs, R.O., De Vreede, T., De Vreede, G.-J., Elkins, A., others, 2020. Machines as teammates: a research agenda on AI in team collaboration. Inf. Manag. 57, 103174.
Stepanenko, V., Liubko, I., 2020. Disaster tweets. Disaster Tweets. Kaggle. https://doi.org/10.34740/KAGGLE/DSV/1640141.
Stephens, K.K., Harris, A.G., Hughes, A.L., Montagnolo, C.E., Nader, K., Stevens, S.A., et al., 2023. Human-AI teaming during an ongoing disaster: how scripts around training and feedback reveal this is a form of human-machine communication. Human-Machine Communication 6 (1), 5.
Suresh, K., 2021. Bitcoin tweets. Bitcoin Tweets. Kaggle.
Tarboton, D.G., 1997. A new method for the determination of flow directions and upslope areas in grid digital elevation models. Water Resour. Res. 33 (2), 309–319.
Wallach, D.A., 2018. Financial tweets. Financial Tweets. Kaggle. Retrieved from. https://www.kaggle.com/datasets/davidwallach/financial-tweets.
Wang, H., Xu, S., Xu, H., Wu, Z., Wang, T., Ma, C., 2023. Rapid prediction of urban flood based on disaster-breeding environment clustering and Bayesian optimized deep learning model in the coastal city. Sustain. Cities Soc. 99, 104898.
Wolshon, B., Urbina, E., Wilmot, C., Levitan, M., 2005. Review of policies and practices for hurricane evacuation. I: transportation planning, preparedness, and response. Nat. Hazards Rev. 6 (3), 129–142.
Wu, Q., 2021. Leafmap: a Python package for interactive mapping and geospatial analysis with minimal coding in a Jupyter environment. J. Open Source Softw. 6 (63), 3414.